01 LC 1 | Two Sum (Easy)

Problem: Return the indices of two distinct elements whose sum is target. Exactly one pair exists; either index order is valid.
Constraints: 2 <= nums.length <= 10^4; -10^9 <= nums[i] <= 10^9; -10^9 <= target <= 10^9; Only one valid answer exists.

```
class Solution {
public:
  vector<int> twoSum(vector<int>& nums, int
    target) {
    unordered_map<int, int> d;
    for (int i = 0;; ++i) {
      int x = nums[i];
      int y = target - x;
      if (d.contains(y)) {
        return {d[y], i};
      }
      d[x] = i;
    }
  }
};
```

04 LC 242 | Valid Anagram (Easy)

Problem: Return whether s and t are anagrams: they contain exactly the same letters with the same multiplicities.
Constraints: 1 <= s.length, t.length <= 5 * 10^4; s and t consist of lowercase English letters.

```
class Solution {
public:
  bool isAnagram(string s, string t) {
    if (s.size() != t.size()) {
      return false;
    }
    vector<int> cnt(26);
    for (int i = 0; i < s.size(); ++i) {
      ++cnt[s[i] - 'a'];
      --cnt[t[i] - 'a'];
    }
    return all_of(cnt.begin(), cnt.end(), [](int
      x) { return x == 0; });
  }
};
```

16 LC 202 | Happy Number (Easy)

Problem: Repeatedly replace positive n with the sum of its squared decimal digits. Return true if this process reaches 1; return false if it enters a different cycle.
Constraints: 1 <= n <= 2^31 - 1

```
class Solution {
public:
  bool isHappy(int n) {
    unordered_set<int> vis;
    while (n != 1 && !vis.count(n)) {
      vis.insert(n);
      int x = 0;
      for (; n; n /= 10) {
        x += (n % 10) * (n % 10);
      }
      n = x;
    }
    return n == 1;
  }
};
```

02 LC 49 | Group Anagrams (Medium)

Problem: Group strings that are anagrams: their characters have identical frequencies. Groups and their contents may be returned in any order.
Constraints: 1 <= strs.length <= 10^4; 0 <= strs[i].length <= 100; strs[i] consists of lowercase English letters.

```
class Solution {
public:
  vector<vector<string>>
    groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> d;
    for (auto& s : strs) {
      int cnt[26] = {0};
      for (auto& c : s) ++cnt[c - 'a'];
      string k;
      for (int i = 0; i < 26; ++i) {
        if (cnt[i]) {
          k += 'a' + i;
          k += to_string(cnt[i]);
        }
      }
      d[k].emplace_back(s);
    }
    vector<vector<string>> ans;
    for (auto& [_, v] : d) ans.emplace_back(v);
    return ans;
  }
};
```

07 LC 560 | Subarray Sum Equals K (Medium)

Problem: Return the number of nonempty contiguous subarrays of nums whose sum equals k.
Constraints: 1 <= nums.length <= 2 * 10^4; -1000 <= nums[i] <= 1000; -10^7 <= k <= 10^7

```
class Solution {
public:
  int subarraySum(vector<int>& nums, int k) {
    unordered_map<int, int> cnt{{0, 1}};
    int ans = 0, s = 0;
    for (int x : nums) {
      s += x;
      ans += cnt[s - k];
      ++cnt[s];
    }
    return ans;
  }
};
```

33 LC 525 | Contiguous Array (Medium)

Problem: Given a binary array nums, return the maximum length of a contiguous subarray with an equal number of 0 and 1.
Constraints: 1 <= nums.length <= 10^5; nums[i] is either 0 or 1.

```
class Solution {
public:
  int findMaxLength(vector<int>& nums) {
    unordered_map<int, int> d{{0, -1}};
    int ans = 0, s = 0;
    for (int i = 0; i < nums.size(); ++i) {
      s += nums[i] ? 1 : -1;
      if (d.contains(s)) {
        ans = max(ans, i - d[s]);
      } else {
        d[s] = i;
      }
    }
    return ans;
  }
};
```

03 LC 217 | Contains Duplicate (Easy)
Problem: Return true if any value occurs at least twice in nums; otherwise return false.
Constraints: 1 <= nums.length <= 10^5; -10^9 <= nums[i] <= 10^9

```
class Solution {
public:
  bool containsDuplicate(vector<int>& nums) {
    unordered_set<int> s(nums.begin(),
      nums.end());
    return s.size() < nums.size();
  }
};
```

45 LC 609 | Find Duplicate File in System (Medium)
Problem: Each string describes a directory followed by file names and parenthesized file contents. Return groups of full file paths whose files have identical content; omit files with no duplicate. Groups and paths may be in any order.
Constraints: 1 <= paths.length <= 2 * 10^4; 1 <= paths[i].length <= 3000; 1 <= sum(paths[i].length) <= 5 * 10^5; paths[i] consist of English letters, digits, '/', '.', '(', ')', and ' '.; You may assume no files or directories share the same name in the same directory.; You may assume each given directory info represents a unique directory. A single blank space separates the directory path and file info.

```
class Solution {
public:
  vector<vector<string>>
    findDuplicate(vector<string>& paths) {
    unordered_map<string, vector<string>> d;
    for (auto& p : paths) {
      auto ps = split(p, ' ');
      for (int i = 1; i < ps.size(); ++i) {
        int j = ps[i].find('(');
        auto content = ps[i].substr(j + 1,
          ps[i].size() - j - 2);
        auto name = ps[0] + '/' + ps[i].substr(0,
          j);
        d[content].push_back(name);
      }
    }
    vector<vector<string>> ans;
    for (auto& [_, e] : d) {
      if (e.size() > 1) {
        ans.push_back(e);
      }
    }
    return ans;
  }
  vector<string> split(string& s, char c) {
    vector<string> res;
    stringstream ss(s);
    string t;
    while (getline(ss, t, c)) {
      res.push_back(t);
    }
    return res;
  }
};
```

05 LC 128 | Longest Consecutive Sequence (Medium)
Problem: Return the length of the longest sequence of consecutive integer values present in unsorted nums. The values need not be adjacent in the array. Required runtime is O(n).
Constraints: 0 <= nums.length <= 10^5; -10^9 <= nums[i] <= 10^9

```
class Solution {
public:
  int longestConsecutive(vector<int>& nums) {
    unordered_set<int> s(nums.begin(),
      nums.end());
    int ans = 0;
    for (int x : s) {
      if (!s.contains(x - 1)) {
        int y = x + 1;
        while (s.contains(y)) {
          y++;
        }
        ans = max(ans, y - x);
      }
    }
    return ans;
  }
};
```

12 LC 383 | Ransom Note (Easy)
Problem: Given two strings ransomNote and magazine, return true if ransomNote can be constructed by using the letters from magazine and false otherwise. Each letter in magazine can only be used once in ransomNote.
Constraints: 1 <= ransomNote.length, magazine.length <= 10^5; ransomNote and magazine consist of lowercase English letters.

```
class Solution {
public:
  bool canConstruct(string ransomNote, string
    magazine) {
    int cnt[26]{};
    for (char& c : magazine) {
      ++cnt[c - 'a'];
    }
    for (char& c : ransomNote) {
      if (--cnt[c - 'a'] < 0) {
        return false;
      }
    }
    return true;
  }
};
```

32 LC 594 | Longest Harmonious Subsequence (Easy)
Problem: We define a harmonious array as an array where the difference between its maximum value and its minimum value is exactly 1. Given an integer array nums, return the length of its longest harmonious subsequence among all its possible subsequences.
Constraints: 1 <= nums.length <= 2 * 10^4; -10^9 <= nums[i] <= 10^9

```
class Solution {
public:
  int findLHS(vector<int>& nums) {
    unordered_map<int, int> cnt;
    for (int x : nums) {
      ++cnt[x];
    }
    int ans = 0;
    for (auto& [x, c] : cnt) {
      if (cnt.contains(x + 1)) {
        ans = max(ans, c + cnt[x + 1]);
      }
    }
    return ans;
  }
};
```

06 LC 347 | Top K Frequent Elements (Medium)
Problem: Return the k most frequent distinct values in nums, in any order. The answer set is unique. Required runtime is better than O(n log n).
Constraints: 1 <= nums.length <= 10^5; -10^4 <= nums[i] <= 10^4; k is in the range [1, the number of unique elements in the array].; It is guaranteed that the answer is unique.

```
class Solution {
public:
  vector<int> topKFrequent(vector<int>& nums, int
    k) {
    unordered_map<int, int> frequency;
    for (int x : nums) ++frequency[x];
    vector<vector<int>> buckets(nums.size() + 1);
    for (auto [value, count] : frequency)
      buckets[count].push_back(value);
    vector<int> answer;
    for (int count = nums.size(); count > 0;
      --count)
      for (int value : buckets[count]) {
        answer.push_back(value);
        if (static_cast<int>(answer.size()) == k)
          return answer;
      }
    return answer;
  }
};
```

18 LC 454 | 4Sum II (Medium)
Problem: Given four integer arrays nums1, nums2, nums3, and nums4 all of length n, return the number of tuples (i, j, k, l) such that: - 0 <= i, j, k, l < n - nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0
Constraints: n == nums1.length; n == nums2.length; n == nums3.length; n == nums4.length; 1 <= n <= 200; -2^28 <= nums1[i], nums2[i], nums3[i], nums4[i] <= 2^28

```
class Solution {
public:
  int fourSumCount(vector<int>& nums1,
    vector<int>& nums2, vector<int>& nums3,
    vector<int>& nums4) {
    unordered_map<int, int> cnt;
    for (int a : nums1) {
      for (int b : nums2) {
        ++cnt[a + b];
      }
    }
    int ans = 0;
    for (int c : nums3) {
      for (int d : nums4) {
        ans += cnt[-(c + d)];
      }
    }
    return ans;
  }
};
```

08 LC 3 | Longest Substring Without Repeating Characters (Medium)
Problem: Return the length of the longest contiguous substring of s containing no repeated characters.
Constraints: 0 <= s.length <= 10^5; s consists of English letters, digits, symbols and spaces.

```
class Solution {
public:
  int lengthOfLongestSubstring(string s) {
    int cnt[128]{};
    int ans = 0, n = s.size();
    for (int l = 0, r = 0; r < n; ++r) {
      ++cnt[s[r]];
      while (cnt[s[r]] > 1) {
        --cnt[s[l++]];
      }
      ans = max(ans, r - l + 1);
    }
    return ans;
  }
};
```

34 LC 523 | Continuous Subarray Sum (Medium)
Problem: Return whether nums has a contiguous subarray of length at least two whose sum is a multiple of positive k. Zero is also a multiple of k.
Constraints: 1 <= nums.length <= 10^5; 0 <= nums[i] <= 10^9; 0 <= sum(nums[i]) <= 2^31 - 1; 1 <= k <= 2^31 - 1

```
class Solution {
public:
  bool checkSubarraySum(vector<int>& nums, int k)
    {
    unordered_map<int, int> d{{0, -1}};
    int s = 0;
    for (int i = 0; i < nums.size(); ++i) {
      s = (s + nums[i]) % k;
      if (!d.contains(s)) {
        d[s] = i;
      } else if (i - d[s] > 1) {
        return true;
      }
    }
    return false;
  }
};
```

39 LC 1679 | Max Number of K-Sum Pairs (Medium)
Problem: You are given an integer array nums and an integer k. In one operation, you can pick two numbers from the array whose sum equals k and remove them from the array. Return the maximum number of operations you can perform on the array.
Constraints: 1 <= nums.length <= 10^5; 1 <= nums[i] <= 10^9; 1 <= k <= 10^9

```
class Solution {
public:
  int maxOperations(vector<int>& nums, int k) {
    unordered_map<int, int> cnt;
    int ans = 0;
    for (int& x : nums) {
      if (cnt[k - x]) {
        --cnt[k - x];
        ++ans;
      } else {
        ++cnt[x];
      }
    }
    return ans;
  }
};
```

09 LC 36 | Valid Sudoku (Medium)
Problem: Check whether a partially filled 9-by-9 Sudoku board has no repeated digit in any row, column, or 3-by-3 box. Ignore '.' cells. Validity does not require that the board be solvable.
Constraints: board.length == 9; board[i].length == 9; board[i][j] is a digit 1-9 or '.'.

```
class Solution {
public:
  bool isValidSudoku(vector<vector<char>>& board)
    {
    vector<vector<bool>> row(9, vector<bool>(9,
      false));
    vector<vector<bool>> col(9, vector<bool>(9,
      false));
    vector<vector<bool>> sub(9, vector<bool>(9,
      false));
    for (int i = 0; i < 9; ++i) {
      for (int j = 0; j < 9; ++j) {
        char c = board[i][j];
        if (c == '.') continue;
        int num = c - '0' - 1;
        int k = i / 3 * 3 + j / 3;
        if (row[i][num] || col[j][num] ||
          sub[k][num]) {
          return false;
        }
        row[i][num] = true;
        col[j][num] = true;
        sub[k][num] = true;
      }
    }
    return true;
  }
};
```

23 LC 380 | Insert Delete GetRandom O(1) (Medium)
Problem: Implement RandomizedSet with insert(val) and remove(val), returning whether membership changed, and getRandom(), selecting each currently stored value with equal probability. All operations require average O(1) time. getRandom is called only when nonempty.
Constraints: -2^31 <= val <= 2^31 - 1; At most 2 * 10^5 calls will be made to insert, remove, and getRandom.; There will be at least one element in the data structure when getRandom is called.

```
class RandomizedSet {
  vector<int> values;
  unordered_map<int, size_t> position;
  mt19937 engine{random_device{}()};
public:
  RandomizedSet() = default;
  bool insert(int val) {
    if (position.contains(val)) return false;
    position[val] = values.size();
    values.push_back(val);
    return true;
  }
  bool remove(int val) {
    auto it = position.find(val);
    if (it == position.end()) return false;
    size_t index = it->second;
    values[index] = values.back();
    position[values[index]] = index;
    values.pop_back();
    position.erase(it);
    return true;
  }
  int getRandom() {
    return
      values[uniform_int_distribution<size_t>(0,
      values.size() - 1)(engine)];
  }
};
```

10 LC 205 | Isomorphic Strings (Easy)
Problem: Given two strings s and t, determine if they are isomorphic. Two strings s and t are isomorphic if the characters in s can be replaced to get t. All occurrences of a character must be replaced with another character while preserving the order of characters. No two characters may map to the same character, but a character may map to itself.
Constraints: 1 <= s.length <= 5 * 10^4; t.length == s.length; s and t consist of any valid ascii character.

```
class Solution {
public:
  bool isIsomorphic(string s, string t) {
    int d1[256]{};
    int d2[256]{};
    int n = s.size();
    for (int i = 0; i < n; ++i) {
      char a = s[i], b = t[i];
      if (d1[a] != d2[b]) {
        return false;
      }
      d1[a] = d2[b] = i + 1;
    }
    return true;
  }
};
```

15 LC 349 | Intersection of Two Arrays (Easy)
Problem: Given two integer arrays nums1 and nums2, return an array of their intersection. Each element in the result must be unique and you may return the result in any order.
Constraints: 1 <= nums1.length, nums2.length <= 1000; 0 <= nums1[i], nums2[i] <= 1000

```
class Solution {
public:
  vector<int> intersection(vector<int>& nums1,
    vector<int>& nums2) {
    bool s[1001];
    memset(s, false, sizeof(s));
    for (int x : nums1) {
      s[x] = true;
    }
    vector<int> ans;
    for (int x : nums2) {
      if (s[x]) {
        ans.push_back(x);
        s[x] = false;
      }
    }
    return ans;
  }
};
```

25 LC 706 | Design HashMap (Easy)
Problem: Implement MyHashMap without built-in hash-table structures: put(key,value) inserts or updates, get(key) returns its value or -1, and remove(key) removes the mapping if present.
Constraints: 0 <= key, value <= 10^6; At most 10^4 calls will be made to put, get, and remove.

```
class MyHashMap {
public:
  int data[1000001];
  MyHashMap() {
    memset(data, -1, sizeof data);
  }
  void put(int key, int value) {
    data[key] = value;
  }
  int get(int key) {
    return data[key];
  }
  void remove(int key) {
    data[key] = -1;
  }
};
```

11 LC 290 | Word Pattern (Easy)
Problem: Return whether pattern characters and space-separated words in s correspond bijectively: equal characters map to equal words, and distinct characters map to distinct words.
Constraints: 1 <= pattern.length <= 300; pattern contains only lower-case English letters.; 1 <= s.length <= 3000; s contains only lowercase English letters and spaces ' '.; s does not contain any leading or trailing spaces.; All the words in s are separated by a single space.

```
class Solution {
public:
  bool wordPattern(string pattern, string s) {
    istringstream is(s);
    vector<string> ws;
    while (is >> s) {
      ws.push_back(s);
    }
    if (pattern.size() != ws.size()) {
      return false;
    }
    unordered_map<char, string> d1;
    unordered_map<string, char> d2;
    for (int i = 0; i < ws.size(); ++i) {
      char a = pattern[i];
      string b = ws[i];
      if ((d1.count(a) && d1[a] != b) ||
        (d2.count(b) && d2[b] != a)) {
        return false;
      }
      d1[a] = b;
      d2[b] = a;
    }
    return true;
  }
};
```

21 LC 76 | Minimum Window Substring (Hard)
Problem: Return the shortest substring of s containing every character of t, including its required multiplicity. Return an empty string if none exists. The answer, when present, is unique.
Constraints: m == s.length; n == t.length; 1 <= m, n <= 10^5; s and t consist of uppercase and lowercase English letters.

```
class Solution {
public:
  string minWindow(string s, string t) {
    vector<int> need(128, 0);
    vector<int> window(128, 0);
    for (char c : t) {
      ++need[c];
    }
    int m = s.length(), n = t.length();
    int k = -1, mi = m + 1, cnt = 0;
    for (int l = 0, r = 0; r < m; ++r) {
      char c = s[r];
      if (++window[c] <= need[c]) {
        ++cnt;
      }
      while (cnt == n) {
        if (r - l + 1 < mi) {
          mi = r - l + 1;
          k = l;
        }
        c = s[l];
        if (window[c] <= need[c]) {
          --cnt;
        }
        --window[c];
        ++l;
      }
    }
    return k < 0 ? "" : s.substr(k, mi);
  }
};
```

13 LC 387 | First Unique Character in a String (Easy)
Problem: Given a string s, find the first non-repeating character in it and return its index. If it does not exist, return -1.
Constraints: 1 <= s.length <= 10^5; s consists of only lowercase English letters.

```
class Solution {
public:
  int firstUniqChar(string s) {
    int cnt[26]{};
    for (char& c : s) {
      ++cnt[c - 'a'];
    }
    int n = s.size();
    for (int i = 0; i < n; ++i) {
      if (cnt[s[i] - 'a'] == 1) {
        return i;
      }
    }
    return -1;
  }
};
```

17 LC 219 | Contains Duplicate II (Easy)
Problem: Given an integer array nums and an integer k, return true if there are two distinct indices i and j in the array such that nums[i] == nums[j] and abs(i - j) <= k.
Constraints: 1 <= nums.length <= 10^5; -10^9 <= nums[i] <= 10^9; 0 <= k <= 10^5

```
class Solution {
public:
  bool containsNearbyDuplicate(vector<int>& nums,
    int k) {
    unordered_map<int, int> d;
    for (int i = 0; i < nums.size(); ++i) {
      if (d.count(nums[i]) && i - d[nums[i]] <= k)
        {
        return true;
      }
      d[nums[i]] = i;
    }
    return false;
  }
};
```

40 LC 1207 | Unique Number of Occurrences (Easy)
Problem: Given an array of integers arr, return true if the number of occurrences of each value in the array is unique or false otherwise.
Constraints: 1 <= arr.length <= 1000; -1000 <= arr[i] <= 1000

```
class Solution {
public:
  bool uniqueOccurrences(vector<int>& arr) {
    unordered_map<int, int> cnt;
    for (int& x : arr) {
      ++cnt[x];
    }
    unordered_set<int> vis;
    for (auto& [_, v] : cnt) {
      if (vis.count(v)) {
        return false;
      }
      vis.insert(v);
    }
    return true;
  }
};
```

14 LC 350 | Intersection of Two Arrays II (Easy)
Problem: Given two integer arrays nums1 and nums2, return an array of their intersection. Each element in the result must appear as many times as it shows in both arrays and you may return the result in any order.
Constraints: 1 <= nums1.length, nums2.length <= 1000; 0 <= nums1[i], nums2[i] <= 1000

```
class Solution {
public:
  vector<int> intersect(vector<int>& nums1,
    vector<int>& nums2) {
    unordered_map<int, int> cnt;
    for (int x : nums1) {
      ++cnt[x];
    }
    vector<int> ans;
    for (int x : nums2) {
      if (cnt[x]-- > 0) {
        ans.push_back(x);
      }
    }
    return ans;
  }
};
```

37 LC 1248 | Count Number of Nice Subarrays (Medium)
Problem: Given an array of integers nums and an integer k. A continuous subarray is called nice if there are k odd numbers on it. Return the number of nice sub-arrays.
Constraints: 1 <= nums.length <= 50000; 1 <= nums[i] <= 10^5; 1 <= k <= nums.length

```
class Solution {
public:
  int numberOfSubarrays(vector<int>& nums, int k)
    {
    int n = nums.size();
    vector<int> cnt(n + 1);
    cnt[0] = 1;
    int ans = 0, t = 0;
    for (int& v : nums) {
      t += v & 1;
      if (t - k >= 0) {
        ans += cnt[t - k];
      }
      cnt[t]++;
    }
    return ans;
  }
};
```

38 LC 532 | K-diff Pairs in an Array (Medium)
Problem: Given an array of integers nums and an integer k, return the number of unique k-diff pairs in the array. A k-diff pair is an integer pair (nums[i], nums[j]), where the following are true: - 0 <= i, j < nums.length - i != j - |nums[i] - nums[j]| == k Notice that |val| denotes the absolute value of val.
Constraints: 1 <= nums.length <= 10^4; -10^7 <= nums[i] <= 10^7; 0 <= k <= 10^7

```
class Solution {
public:
  int findPairs(vector<int>& nums, int k) {
    unordered_set<int> ans, vis;
    for (int x : nums) {
      if (vis.count(x - k)) {
        ans.insert(x - k);
      }
      if (vis.count(x + k)) {
        ans.insert(x);
      }
      vis.insert(x);
    }
    return ans.size();
  }
};
```

19 LC 438 | Find All Anagrams in a String (Medium)
Problem: Return every starting index of a substring of s that is an anagram of p. Indices may be returned in any order.
Constraints: 1 <= s.length, p.length <= 3 * 10^4; s and p consist of lowercase English letters.

```
class Solution {
public:
  vector<int> findAnagrams(string s, string p) {
    int m = s.size(), n = p.size();
    vector<int> ans;
    if (m < n) {
      return ans;
    }
    vector<int> cnt1(26);
    for (char& c : p) {
      ++cnt1[c - 'a'];
    }
    vector<int> cnt2(26);
    for (int i = 0; i < n - 1; ++i) {
      ++cnt2[s[i] - 'a'];
    }
    for (int i = n - 1; i < m; ++i) {
      ++cnt2[s[i] - 'a'];
      if (cnt1 == cnt2) {
        ans.push_back(i - n + 1);
      }
      --cnt2[s[i - n + 1] - 'a'];
    }
    return ans;
  }
};
```

26 LC 981 | Time Based Key-Value Store (Medium)
Problem: Implement TimeMap.set(key,value,timestamp) and get(key,timestamp). get returns the value with the greatest stored timestamp not exceeding the query, or an empty string if none exists. Set timestamps are strictly increasing.
Constraints: 1 <= key.length, value.length <= 100; key and value consist of lowercase English letters and digits.; 1 <= timestamp <= 10^7; All the timestamps timestamp of set are strictly increasing.; At most 2 * 10^5 calls will be made to set and get.

```
class TimeMap {
public:
  TimeMap() {
  }
  void set(string key, string value, int
    timestamp) {
    ktv[key].emplace_back(timestamp, value);
  }
  string get(string key, int timestamp) {
    auto& pairs = ktv[key];
    pair<int, string> p = {timestamp,
      string({127})};
    auto i = upper_bound(pairs.begin(),
      pairs.end(), p);
    return i == pairs.begin() ? "" : (i -
      1)->second;
  }
private:
  unordered_map<string, vector<pair<int, string>>>
    ktv;
};
```

20 LC 567 | Permutation in String (Medium)

Problem: Return whether s2 contains a contiguous substring that is a permutation of s1.

Constraints: 1 <= s1.length, s2.length <= 10^4; s1 and s2 consist of lowercase English letters.

```
class Solution {
public:
  bool checkInclusion(string s1, string s2) {
    int need = 0;
    int cnt[26]{};
    for (char c : s1) {
      if (++cnt[c - 'a'] == 1) {
        ++need;
      }
    }
    int m = s1.size(), n = s2.size();
    for (int i = 0; i < n; ++i) {
      int c = s2[i] - 'a';
      if (--cnt[c] == 0) {
        --need;
      }
      if (i >= m) {
        c = s2[i - m] - 'a';
        if (++cnt[c] == 1) {
          ++need;
        }
      }
      if (need == 0) {
        return true;
      }
    }
    return false;
  }
};
```

27 LC 1396 | Design Underground System (Medium)

Problem: Implement UndergroundSystem.checkIn(id,station,t), checkOut(id,station,t), and getAverageTime(start,end). A checkout completes the passenger's active trip. Return the mean duration of completed direct trips for that ordered station pair; queried pairs have completed trips.

Constraints: 1 <= id, t <= 10^6; 1 <= stationName.length, startStation.length, endStation.length <= 10; All strings consist of uppercase and lowercase English letters and digits.; There will be at most 2 * 10^4 calls in total to checkIn, checkOut, and getAverageTime.; Answers within 10^-5 of the actual value will be accepted.

```
class UndergroundSystem {
public:
  UndergroundSystem() {
  }
  void checkIn(int id, string stationName, int t)
    {
    ts[id] = {stationName, t};
  }
  void checkOut(int id, string stationName, int t)
    {
    auto [station, t0] = ts[id];
    auto key = station + "-" + stationName;
    auto [tot, cnt] = d[key];
    d[key] = {tot + t - t0, cnt + 1};
  }
  double getAverageTime(string startStation,
    string endStation) {
    auto [tot, cnt] = d[startStation + "-" +
      endStation];
    return (double) tot / cnt;
  }
private:
  unordered_map<int, pair<string, int>> ts;
  unordered_map<string, pair<int, int>> d;
};
```

22 LC 146 | LRU Cache (Medium)

Problem: Implement LRUCache(capacity): get(key) returns its value or -1; put(key,value) inserts or updates. When insertion exceeds capacity, evict the least recently used entry. Both operations require average O(1) time.

Constraints: 1 <= capacity <= 3000; 0 <= key <= 10^4; 0 <= value <= 10^5; At most 2 * 10^5 calls will be made to get and put.

```
class LRUCache {
  size_t capacity;
  list<pair<int, int>> entries;
  unordered_map<int, list<pair<int,
    int>>::iterator> location;
public:
  LRUCache(int capacity) : capacity(capacity) {}
  LRUCache(const LRUCache&) = delete;
  LRUCache& operator=(const LRUCache&) = delete;
  int get(int key) {
    auto it = location.find(key);
    if (it == location.end()) return -1;
    entries.splice(entries.begin(), entries,
      it->second);
    return it->second->second;
  }
  void put(int key, int value) {
    if (auto it = location.find(key); it !=
      location.end()) {
      it->second->second = value;
      entries.splice(entries.begin(), entries,
        it->second);
      return;
    }
    if (capacity == 0) return;
    if (entries.size() == capacity) {
      location.erase(entries.back().first);
      entries.pop_back();
    }
    entries.emplace_front(key, value);
    location[key] = entries.begin();
  }
};
```

48 LC 133 | Clone Graph (Medium)

Problem: Return a deep copy of the connected undirected graph reachable from the supplied node. Each node has a value and neighbor list. Clone every node/link; return null for an empty graph.

Constraints: The number of nodes in the graph is in the range [0, 100].; 1 <= Node.val <= 100; Node.val is unique for each node.; There are no repeated edges and no self-loops in the graph.; The Graph is connected and all nodes can be visited starting from the given node.

```
class Solution {
public:
  Node* cloneGraph(Node* node) {
    unordered_map<Node*, Node*> g;
    auto dfs = [&](auto&& dfs, Node* node) ->
      Node* {
      if (!node) {
        return nullptr;
      }
      if (g.contains(node)) {
        return g[node];
      }
      Node* cloned = new Node(node->val);
      g[node] = cloned;
      for (auto& nxt : node->neighbors) {
        cloned->neighbors.push_back(dfs(dfs,
          nxt));
      }
      return cloned;
    };
    return dfs(dfs, node);
  }
};
```

24 LC 705 | Design HashSet (Easy)

Problem: Design a HashSet without using any built-in hash table libraries. Implement MyHashSet class: - void add(key) Inserts the value key into the HashSet. - bool contains(key) Returns whether the value key exists in the HashSet or not. - void remove(key) Removes the value key in the HashSet. If key does not exist in the HashSet, do nothing.

Constraints: 0 <= key <= 10^6; At most 10^4 calls will be made to add, remove, and contains.

```
class MyHashSet {
public:
  bool data[1000001];
  MyHashSet() {
    memset(data, false, sizeof data);
  }
  void add(int key) {
    data[key] = true;
  }
  void remove(int key) {
    data[key] = false;
  }
  bool contains(int key) {
    return data[key];
  }
};
```

31 LC 409 | Longest Palindrome (Easy)

Problem: Given a string s which consists of lowercase or uppercase letters, return the length of the longest palindrome that can be built with those letters. Letters are case sensitive, for example, "Aa" is not considered a palindrome.

Constraints: 1 <= s.length <= 2000; s consists of lowercase and/or uppercase English letters only.

```
class Solution {
public:
  int longestPalindrome(string s) {
    int cnt[128]{};
    for (char c : s) {
      ++cnt[c];
    }
    int ans = 0;
    for (int v : cnt) {
      ans += v / 2 * 2;
    }
    ans += ans < s.size();
    return ans;
  }
};
```

46 LC 187 | Repeated DNA Sequences (Medium)

Problem: The DNA sequence is composed of a series of nucleotides abbreviated as 'A', 'C', 'G', and 'T'. - For example, "ACGAATTCCG" is a DNA sequence. When studying DNA, it is useful to identify repeated sequences within the DNA. Given a string s that represents a DNA sequence, return all the 10-letter-long sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in any order.

Constraints: 1 <= s.length <= 10^5; s[i] is either 'A', 'C', 'G', or 'T'.

```
class Solution {
public:
  vector<string> findRepeatedDnaSequences(string
    s) {
    unordered_map<string, int> cnt;
    vector<string> ans;
    for (int i = 0, n = s.size() - 10 + 1; i < n;
      ++i) {
      auto t = s.substr(i, 10);
      if (++cnt[t] == 2) {
        ans.emplace_back(t);
      }
    }
    return ans;
  }
};
```

28 LC 460 | LFU Cache (Hard)

Problem: Implement LFUCache(capacity). get(key) returns its value or -1; put(key,value) inserts or updates. At capacity, evict the least frequently used key, breaking ties by least recent use. New entries have frequency 1; successful gets and updates increment frequency. Both operations require average O(1) time.

Constraints: 1 <= capacity <= 10^4; 0 <= key <= 10^5; 0 <= value <= 10^9; At most 2 * 10^5 calls will be made to get and put.

```
class LFUCache {
  struct Entry { int value, frequency;
    list<int>::iterator position; };
  int capacity, minimum = 0;
  unordered_map<int, Entry> entries;
  unordered_map<int, list<int>> buckets;
  void touch(unordered_map<int, Entry>::iterator
    it) {
    int key = it->first, frequency =
      it->second.frequency;
    buckets.try_emplace(frequency + 1);
    auto& from = buckets.at(frequency);
    auto& to = buckets.at(frequency + 1);
    to.splice(to.begin(), from,
      it->second.position);
    it->second.frequency++;
    it->second.position = to.begin();
    if (from.empty()) {
      buckets.erase(frequency);
      if (minimum == frequency) ++minimum;
    }
  }
public:
  LFUCache(int capacity) : capacity(capacity) {}
  LFUCache(const LFUCache&) = delete;
  LFUCache& operator=(const LFUCache&) = delete;
  int get(int key) {
    auto it = entries.find(key);
    if (it == entries.end()) return -1;
    touch(it);
    return it->second.value;
  }
  void put(int key, int value) {
    if (!capacity) return;
    if (auto it = entries.find(key); it !=
      entries.end()) {
      it->second.value = value;
      touch(it);
      return;
    }
    if (static_cast<int>(entries.size()) ==
      capacity) {
      auto& least = buckets.at(minimum);
      entries.erase(least.back());
      least.pop_back();
      if (least.empty()) buckets.erase(minimum);
    }
    auto& first = buckets[1];
    first.push_front(key);
    entries.emplace(key, Entry{value, 1,
      first.begin()});
    minimum = 1;
  }
};
```

29 LC 692 | Top K Frequent Words (Medium)
Problem: Given an array of strings words and an integer k, return the k most frequent strings. Return the answer sorted by the frequency from highest to lowest. Sort the words with the same frequency by their lexicographical order.
Constraints: 1 <= words.length <= 500; 1 <= words[i].length <= 10; words[i] consists of lowercase English letters.; k is in the range [1, The number of unique words[i]]

```
class Solution {
public:
  vector<string> topKFrequent(vector<string>&
    words, int k) {
    unordered_map<string, int> cnt;
    for (const auto& w : words) {
      ++cnt[w];
    }
    vector<string> ans;
    for (const auto& [w, _] : cnt) {
      ans.push_back(w);
    }
    ranges::sort(ans, [&](const string& a, const
      string& b) {
      return cnt[a] > cnt[b] || (cnt[a] == cnt[b]
        && a < b);
    });
    ans.resize(k);
    return ans;
  }
};
```

30 LC 451 | Sort Characters By Frequency (Medium)
Problem: Given a string s, sort it in decreasing order based on the frequency of the characters. The frequency of a character is the number of times it appears in the string. Return the sorted string. If there are multiple answers, return any of them.
Constraints: 1 <= s.length <= 5 * 10^5; s consists of uppercase and lowercase English letters and digits.

```
class Solution {
public:
  string frequencySort(string s) {
    unordered_map<char, int> cnt;
    for (char& c : s) {
      ++cnt[c];
    }
    vector<char> cs;
    for (auto& [c, _] : cnt) {
      cs.push_back(c);
    }
    ranges::sort(cs, [&](char& a, char& b) {
      return cnt[a] > cnt[b];
    });
    string ans;
    for (char& c : cs) {
      ans += string(cnt[c], c);
    }
    return ans;
  }
};
```

35 LC 974 | Subarray Sums Divisible by K (Medium)
Problem: Given an integer array nums and an integer k, return the number of non-empty subarrays that have a sum divisible by k. A subarray is a contiguous part of an array.
Constraints: 1 <= nums.length <= 3 * 10^4; -10^4 <= nums[i] <= 10^4; 2 <= k <= 10^4

```
class Solution {
public:
  int subarraysDivByK(vector<int>& nums, int k) {
    unordered_map<int, int> cnt{{0, 1}};
    int ans = 0, s = 0;
    for (int& x : nums) {
      s = ((s + x) % k + k) % k;
      ans += cnt[s]++;
    }
    return ans;
  }
};
```

36 LC 930 | Binary Subarrays With Sum (Medium)
Problem: Given a binary array nums and an integer goal, return the number of non-empty subarrays with a sum goal. A subarray is a contiguous part of the array.
Constraints: 1 <= nums.length <= 3 * 10^4; nums[i] is either 0 or 1.; 0 <= goal <= nums.length

```
class Solution {
public:
  int numSubarraysWithSum(vector<int>& nums, int
    goal) {
    int i1 = 0, i2 = 0, s1 = 0, s2 = 0, j = 0, ans
      = 0;
    int n = nums.size();
    while (j < n) {
      s1 += nums[j];
      s2 += nums[j];
      while (i1 <= j && s1 > goal) s1 -=
        nums[i1++];
      while (i2 <= j && s2 >= goal) s2 -=
        nums[i2++];
      ans += i2 - i1;
      ++j;
    }
    return ans;
  }
};
```

44 LC 1487 | Making File Names Unique (Medium)
Problem: Create folders in input order. Keep a requested name if unused; otherwise append '(k)' using the smallest positive k making the full name unused. Return the actual names in creation order.
Constraints: 1 <= names.length <= 5 * 10^4; 1 <= names[i].length <= 20; names[i] consists of lowercase English letters, digits, and/or round brackets.

```
class Solution {
public:
  vector<string> getFolderNames(vector<string>&
    names) {
    unordered_map<string, int> d;
    for (auto& name : names) {
      int k = d[name];
      if (k) {
        while (d[name + "(" + to_string(k) + ")"])
          {
          k++;
        }
        d[name] = k;
        name += "(" + to_string(k) + ")";
      }
      d[name] = 1;
    }
    return names;
  }
};
```

41 LC 1657 | Determine if Two Strings Are Close (Medium)
Problem: Strings are close if one can become the other by permuting positions and by globally swapping any two characters already present. Return whether word1 and word2 have this relationship.
Constraints: 1 <= word1.length, word2.length <= 10^5; word1 and word2 contain only lowercase English letters.

```
class Solution {
public:
  bool closeStrings(string word1, string word2) {
    int cnt1[26]{};
    int cnt2[26]{};
    for (char& c : word1) {
      ++cnt1[c - 'a'];
    }
    for (char& c : word2) {
      ++cnt2[c - 'a'];
    }
    for (int i = 0; i < 26; ++i) {
      if ((cnt1[i] == 0) != (cnt2[i] == 0)) {
        return false;
      }
    }
    sort(cnt1, cnt1 + 26);
    sort(cnt2, cnt2 + 26);
    return equal(cnt1, cnt1 + 26, cnt2);
  }
};
```

42 LC 2225 | Find Players With Zero or One Losses (Medium)
Problem: Each match lists a winner and loser. Return two sorted lists of players who participated: those with no losses, and those with exactly one loss.
Constraints: 1 <= matches.length <= 10^5; matches[i].length == 2; 1 <= winner_i, loser_i <= 10^5; winner_i != loser_i; All matches[i] are unique.

```
class Solution {
public:
  vector<vector<int>>
    findWinners(vector<vector<int>>& matches) {
    map<int, int> cnt;
    for (auto& e : matches) {
      if (!cnt.contains(e[0])) {
        cnt[e[0]] = 0;
      }
      ++cnt[e[1]];
    }
    vector<vector<int>> ans(2);
    for (auto& [x, v] : cnt) {
      if (v < 2) {
        ans[v].push_back(x);
      }
    }
    return ans;
  }
};
```

43 LC 2364 | Count Number of Bad Pairs (Medium)
Problem: You are given a 0-indexed integer array nums. A pair of indices (i, j) is a bad pair if i < j and j - i != nums[j] - nums[i]. Return the total number of bad pairs in nums.
Constraints: 1 <= nums.length <= 10^5; 1 <= nums[i] <= 10^9

```
class Solution {
public:
  long long countBadPairs(vector<int>& nums) {
    unordered_map<int, int> cnt;
    long long ans = 0;
    for (int i = 0; i < nums.size(); ++i) {
      int x = i - nums[i];
      ans += i - cnt[x]++;
    }
    return ans;
  }
};
```

47 LC 652 | Find Duplicate Subtrees (Medium)
Problem: Given the root of a binary tree, return all duplicate subtrees. For each kind of duplicate subtrees, you only need to return the root node of any one of them. Two trees are duplicate if they have the same structure with the same node values.
Constraints: The number of the nodes in the tree will be in the range [1, 5000]; -200 <= Node.val <= 200

```
class Solution {
  struct Key {
    int value, left, right;
    bool operator==(const Key&) const = default;
  };
  struct Hash {
    size_t operator()(const Key& key) const {
      size_t h = hash<int>{}(key.value);
      h ^= hash<int>{}(key.left) + 0x9e3779b9U +
        (h << 6) + (h >> 2);
      h ^= hash<int>{}(key.right) + 0x9e3779b9U +
        (h << 6) + (h >> 2);
      return h;
    }
  };
public:
  vector<TreeNode*>
    findDuplicateSubtrees(TreeNode* root) {
    unordered_map<Key, int, Hash> intern;
    unordered_map<TreeNode*, int> id;
    vector<int> count(1);
    vector<TreeNode*> result;
    vector<pair<TreeNode*, bool>> stack;
    if (root) stack.emplace_back(root, false);
    while (!stack.empty()) {
      auto [node, visited] = stack.back();
        stack.pop_back();
      if (!visited) {
        stack.emplace_back(node, true);
        if (node->right)
          stack.emplace_back(node->right, false);
        if (node->left)
          stack.emplace_back(node->left, false);
      } else {
        Key key{node->val, node->left ?
          id.at(node->left) : 0, node->right ?
          id.at(node->right) : 0};
        auto [it, inserted] =
          intern.try_emplace(key, count.size());
        if (inserted) count.push_back(0);
        id[node] = it->second;
        if (++count[it->second] == 2)
          result.push_back(node);
      }
    }
    return result;
  }
};
```

49 LC 721 | Accounts Merge (Medium)

Problem: Each account contains a name followed by emails. Merge accounts connected through shared emails; equal names alone do not identify a person. All accounts of one person share a name. Return each merged name followed by sorted emails, with merged accounts in any order.

Constraints: 1 <= accounts.length <= 1000; 2 <= accounts[i].length <= 10; 1 <= accounts[i][j].length <= 30; accounts[i][0] consists of English letters.; accounts[i][j] (for j > 0) is a valid email.

```
class UnionFind {
public:
  UnionFind(int n) {
    p = vector<int>(n);
    size = vector<int>(n, 1);
    iota(p.begin(), p.end(), 0);
  }
  bool unite(int a, int b) {
    int pa = find(a), pb = find(b);
    if (pa == pb) {
      return false;
    }
    if (size[pa] > size[pb]) {
      p[pb] = pa;
      size[pa] += size[pb];
    } else {
      p[pa] = pb;
      size[pb] += size[pa];
    }
    return true;
  }
  int find(int x) {
    if (p[x] != x) {
      p[x] = find(p[x]);
    }
    return p[x];
  }
private:
  vector<int> p, size;
};
class Solution {
public:
  vector<vector<string>>
    accountsMerge(vector<vector<string>>&
    accounts) {
    int n = accounts.size();
    UnionFind uf(n);
    unordered_map<string, int> d;
    for (int i = 0; i < n; ++i) {
      for (int j = 1; j < accounts[i].size(); ++j)
        {
        const string& email = accounts[i][j];
        if (d.find(email) != d.end()) {
          uf.unite(i, d[email]);
        } else {
          d[email] = i;
        }
      }
    }
    unordered_map<int, set<string>> g;
    for (int i = 0; i < n; ++i) {
      int root = uf.find(i);
      g[root].insert(accounts[i].begin() + 1,
        accounts[i].end());
    }
    vector<vector<string>> ans;
    for (const auto& [root, s] : g) {
      vector<string> emails(s.begin(), s.end());
      emails.insert(emails.begin(),
        accounts[root][0]);
      ans.push_back(emails);
    }
    return ans;
  }
};
```

50 LC 355 | Design Twitter (Medium)

Problem: Implement Twitter: postTweet(userId,tweetId); follow(followerId,followeeId); unfollow(...); getNewsFeed(userId). A feed contains at most the 10 newest tweets from the user and followed users, newest first. Each tweet ID is unique.

Constraints: 1 <= userId, followerId, followeeId <= 500; 0 <= tweetId <= 10^4; All the tweets have unique IDs.; At most 3 * 10^4 calls will be made to postTweet, getNewsFeed, follow, and unfollow.; A user cannot follow himself.

```
class Twitter {
  struct Tweet { long long time; int id; };
  unordered_map<int, vector<Tweet>> tweets;
  unordered_map<int, unordered_set<int>> follows;
  long long clock = 0;
public:
  Twitter() = default;
  void postTweet(int userId, int tweetId) {
    tweets[userId].push_back({clock++, tweetId});
  }
  vector<int> getNewsFeed(int userId) {
    using Entry = tuple<long long, int, int>;
    vector<Entry> initial;
    auto add = [&](int author) {
      auto it = tweets.find(author);
      if (it != tweets.end() &&
        !it->second.empty()) {
        int i =
          static_cast<int>(it->second.size()) - 1;
        initial.emplace_back(it->second[i].time,
          author, i);
      }
    };
    add(userId);
    if (auto it = follows.find(userId); it !=
      follows.end())
      for (int author : it->second) if (author !=
        userId) add(author);
    priority_queue<Entry> heap(less<Entry>{},
      move(initial));
    vector<int> result;
    while (!heap.empty() && result.size() < 10) {
      auto [time, author, i] = heap.top();
      heap.pop();
      result.push_back(tweets[author][i].id);
      if (i > 0) heap.emplace(tweets[author][i -
        1].time, author, i - 1);
    }
    return result;
  }
  void follow(int followerId, int followeeId) {
    if (followerId != followeeId)
      follows[followerId].insert(followeeId);
  }
  void unfollow(int followerId, int followeeId) {
    if (auto it = follows.find(followerId); it !=
      follows.end()) it->second.erase(followeeId);
  }
};
```